

Розділ 6

Рядки

6.1 Рядок – це послідовність

Рядок – це *послідовність* символів. За допомогою оператора квадратних дужок можна отримати доступ до символів по одному за раз:

```
>>> fruit = 'банан'
>>> letter = fruit[1]
```

Друга інструкція отримує символ в позиції з індексом 1 зі змінної `fruit` і присвоює його змінній `letter`.

Вираз у дужках називається *індексом*. Індекс вказує на символ у послідовності, яка вам потрібна.

Однак можна отримати не те, на що очікували:

```
>>> print(letter)
а
```

Людина звикла, що перша літера слова «банан» – це «б», а не «а». Але у Python індекс – це відступ від початку рядка, а відступ першої літери нульовий.

```
>>> letter = fruit[0]
>>> print(letter)
б
```

Отже, «б» – це 0-а буква («нульова») слова «банан», «а» – 1-а буква («перша»), а «н» – 2-а буква («друга»).

Як індекс можна використовувати будь-який вираз, навіть змінні та оператори, але значення індексу має бути цілим числом. Інакше отримаєте:

РОЗДІЛ 6. РЯДКИ

```
>>> letter = fruit[1.5]
```

TypeError: string indices must be integers

б	а	н	а	н
[0]	[1]	[2]	[3]	[4]

Рисунок 6.1: Індеси рядків

6.2 Отримання довжини рядка за допомогою len

`len()` – це вбудована функція, яка повертає кількість символів у рядку:

```
>>> fruit = 'банан'
```

```
>>> len(fruit)
```

```
5
```

Щоб вивести останню літеру рядка, можна зробити щось подібне:

```
>>> length = len(fruit)
```

```
>>> last = fruit[length]
```

IndexError: string index out of range

Причина виникнення помилки `IndexError` в тому, що у слові «банан» немає літери з індексом 5. Оскільки відлік почався з нуля, то п'ять літер пронумеровані від 0 до 4. Щоб вивести останню літеру, потрібно від довжини відняти 1:

```
>>> last = fruit[length-1]
```

```
>>> print(last)
```

```
н
```

Крім того, можна скористатися від'ємними індексами, які відраховують у зворотному напрямку від кінця рядка. Вираз `fruit[-1]` повертає останню літеру, `fruit[-2]` – передостанню і так до кінця.

6.3 Перебір рядка з використанням циклу

Велика кількість обчислень передбачає опрацювання рядка по одному символу за раз. Часто починають з початку, по черзі обирають кожен символ, виконують з ним певні дії та продовжують до кінця. Таку структуру процесу обробки називають *перебір*. Перебір можна записати за допомогою циклу `while`:

```
index = 0
while index < len(fruit):
    letter = fruit[index]
    print(letter)
    index = index + 1
```

Цей цикл перебирає рядок і виводить кожну літеру окремо. Умова циклу `index < len(fruit)`, тому коли індекс дорівнює довжині рядка, умова хибна і тіло циклу не виконується. Останній символ, до якого отримано доступ – це символ з індексом `len(fruit)-1`, який є останнім символом у рядку.

Вправа 1: Напишіть цикл `while`, який починається з останнього символу рядка і рухається до першого, виводячи кожну літеру окремо, лише в зворотному напрямку.

Інший спосіб написати перебір – за допомогою циклу `for`:

```
for char in fruit:
    print(char)
```

Щоразу при виконанні циклу змінній `char` присвоюється наступний символ у рядку. Цикл продовжується доти, доки символи не закінчаться.

6.4 Зрізи рядків

Фрагмент рядка називається *зрізом*. Вибір фрагмента подібний до вибору символу:

```
>>> s = 'Монті Пайтон'
>>> print(s[0:5])
Монті
```

РОЗДІЛ 6. РЯДКИ

```
>>> print(s[6:12])
```

Пайтон

Оператор `[n:m]` повертає частину рядка від «n-го» до «m-го» символу, включаючи перший символ, але не включаючи останній.

Якщо опустити перший індекс (перед двокрапкою), то фрагмент почнеться з початку рядка. Якщо опустити другий індекс, то фрагмент буде перенесено у кінець рядка:

```
>>> fruit = 'банани'
```

```
>>> fruit[:3]
```

```
'бан'
```

```
>>> fruit[3:]
```

```
'ани'
```

Якщо перший індекс більше або дорівнює другому, результатом буде порожній рядок, поданий у лапках:

```
>>> fruit = 'банани'
```

```
>>> fruit[3:3]
```

```
''
```

У порожньому рядку немає символів, а його довжина – 0, але в іншому він такий самий, як і будь-який звичайний рядок.

Вправа 2: Якщо `fruit` – це рядок, то що означає `fruit[:]`?

6.5 Рядки незмінні

Часто виникає спокуса застосувати оператор у лівій частині присвоювання, щоб змінити символ у рядку. Наприклад:

```
>>> greeting = 'Привіт, світе!'
```

```
>>> greeting[0] = 'Ж'
```

```
TypeError: 'str' object does not support item assignment
```

«Object» (об'єкт) у цьому разі – це рядок, а «item» (елемент) – це символ, який ви намагалися присвоїти. Наразі *об'єкт* – це те саме, що і значення, але пізніше ми доопрацюємо це визначення. *Елемент* – це одне із значень у послідовності.

Причина помилки в тому, що рядки є *незмінними*, а це означає, що не можна змінювати наявний рядок. Єдине, що можливо зробити, – це створити новий рядок, який буде різновидом початкового:

```
>>> greeting = 'Привіт, світе!'
>>> new_greeting = 'Ж' + greeting[1:]
>>> print(new_greeting)
Жривіт, світе!
```

У цьому прикладі до фрагмента привітання додається нова перша літера, це не впливає на вихідний рядок.

6.6 Циклічне виконання та підрахунок

Нижче наведена програма, яка підраховує, скільки разів у рядку зустрічається літера «а»:

```
word = 'банан'
count = 0
    for letter in word:
        if letter == 'a':
            count = count + 1
print(count)
```

Ця програма демонструє ще один зразок обчислень, який називається *лічильником*. Змінна `count` ініціалізується значенням 0, а потім збільшується кожного разу, коли зустрічається літера «а». При виході з циклу змінна `count` міститиме результат – загальну кількість знайдених літер «а».

РОЗДІЛ 6. РЯДКИ

Вправа 3: Імплементуйте цей код у функцію з іменем `count` і узагальніть її таким чином, аби вона приймала рядок і літеру як аргументи.

6.7 Оператор `in`

Слово `in` – це булевий (логічний) оператор, який обробляє два рядки і повертає значення `True`, якщо перший рядок з'являється як підрядок у другому:

```
>>> 'a' in 'банан'
True
>>> 'зерно' in 'банан'
False
```

6.8 Порівняння рядків

Інструкції порівняння працюють з рядками, щоб перевірити, чи є два рядки рівними:

```
if word == 'банан':
    print('Ну що, банани.')
```

Інші дії порівняння корисні для розміщення слів за алфавітом:

```
if word < 'банан':
    print('Вказане слово,' + word + ', знаходиться перед банан.')
elif word > 'банан':
    print('Вказане слово,' + word + ', знаходиться після банан.')
else:
    print('Ну що, банани.')
```

Python не сприймає великі та малі літери так само як люди. Усі великі літери йдуть перед усіма малими, тому:

Вказане слово, Ананас, знаходиться перед банан.

Найпоширенішим способом розв'язку цієї проблеми є конвертація рядків до стандартного формату, наприклад, перетворення на всі малі літери, перед здійсненням порівняння. Не забувайте про це на випадок, якщо вам доведеться захищатися від людини, озброєної Ананасом.

6.9 Методи рядків

Рядки – це приклад *об'єктів* Python. Об'єкт містить як дані (власне, рядок), так і методи, які насправді є функціями, вбудованими в об'єкт і доступними для будь-якого об'єкта.

У Python є функція `dir`, яка відображає список методів, доступних для об'єкта. Функція `type` показує тип об'єкта, а функція `dir` – доступні методи.

```
>>> stuff = 'Привіт світе'
>>> type(stuff)
<class 'str'>
>>> dir(stuff)
[... 'capitalize', 'casefold', 'center', 'count', 'encode',
'endswith', 'expandtabs', 'find', 'format', 'format_map',
'index', 'isalnum', 'isalpha', 'isdecimal', 'isdigit',
'identifier', 'islower', 'isnumeric', 'isprintable',
'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower',
'lstrip', 'maketrans', 'partition', 'replace', 'rfind',
'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip',
'split', 'splitlines', 'startswith', 'strip', 'swapcase',
'title', 'translate', 'upper', 'zfill']
>>> help(str.capitalize)
Help on method_descriptor:

capitalize(self, /)
    Return a capitalized version of the string.
```

РОЗДІЛ 6. РЯДКИ

More specifically, make the first character have upper case and the rest lower case.

```
>>>
```

Попри те, що функція `dir` містить список, а за допомогою довідки ви можете отримати просту документацію, кращим джерелом інформації про методи рядків є сайт

<https://docs.python.org/library/stdtypes.html#string-methods>.

Виклик *методу* схожий на виклик функції (він приймає аргументи і повертає значення), однак тут відрізняється синтаксис. Метод слід викликати, додавши ім'я методу до імені змінної, використовуючи крапку для відокремлення.

Наприклад, метод `upper` приймає рядок і повертає новий з усіма великими літерами:

Замість синтаксису функції `upper(word)` використовується синтаксис методу `word.upper()`.

```
>>> word = 'банан'
>>> new_word = word.upper()
>>> print(new_word)
БАНАН
```

Ця форма запису з крапкою встановлює назву методу, `upper`, та назву рядка, до якого застосовується метод, `word`. Порожні дужки вказують на те, що цей метод не приймає аргументи.

Виклик методу так і називається *виклик*; у цьому випадку можна сказати, що ми викликаємо метод `upper` для рядкової змінної `word`.

Наприклад, існує метод пошуку рядка `find`, який шукає позицію одного рядка в іншому:

```
>>> word = 'банани'
>>> index = word.find('a')
```



```
>>> print(index)
```

```
1
```

У цьому прикладі ми викликаємо `find` для змінної `word` і задаємо літеру, яку шукаємо, як параметр.

Метод `find` знаходить як підрядки, так і символи:

```
>>> word.find('на')
```

```
2
```

Другим аргументом може бути індекс, з якого слід починати:

```
>>> word.find('ни', 3)
```

```
4
```

Поширеним завданням є видалення пробілів (пропусків, табуляцій або нових рядків) з початку і кінця рядка за допомогою методу `strip`:

```
>>> line = ' Поїхали '
```

```
>>> line.strip()
```

```
'Поїхали'
```

Деякі методи, як-от `startswith`, повертають булеві значення.

```
>>> line = 'Гарного дня'
```

```
>>> line.startswith('Гарного')
```

```
True
```

```
>>> line.startswith('г')
```

```
False
```

Зауважте, що для `startswith` необхідно, щоб регістр збігався, тому іноді потрібно взяти рядок і змінити його на нижній регістр за допомогою методу `lower`, перш ніж виконувати перевірку.

```
>>> line = 'Гарного дня'
```

```
>>> line.startswith('г')
```

```
False
```

```
>>> line.lower()
```

```
'гарного дня'
```

```
>>> line.lower().startswith('г')
```

РОЗДІЛ 6. РЯДКИ

True

В останньому прикладі було викликано метод `lower`, після чого використано метод `startswith`, щоб перевірити, чи починається отриманий рядок з літери «г» у нижньому регістрі. Якщо дотримуватися порядку виклику методів, то в одному виразі можна викликати кілька методів.

Вправа 4: Існує метод `count` для роботи з рядками, який схожий на функцію з попередньої вправи. Прочитайте документацію до цього методу за посиланням:

<https://docs.python.org/library/stdtypes.html#string-methods>

Напишіть програму, яка підраховує, скільки разів літера «а» зустрічається у слові «банан».

6.10 Парсинг рядків

Нерідко виникає потреба розглянути рядок і знайти в ньому підрядок. Наприклад, якщо задано кілька рядків, які мають такий формат:

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

а з кожного рядка необхідно витягнути лише другу половину адреси (наприклад, `uct.ac.za`). Це можна зробити за допомогою методу `find` і зрізу рядка.

Спочатку слід знайти позицію символу `@` у рядку. Потім шукаємо позицію першого пробілу *після* символу `@`. А далі потрібно зробити зріз рядка, щоб витягти потрібний фрагмент.

```
>>> data = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'
>>> atpos = data.find('@')
>>> print(atpos)
21
>>> spos = data.find(' ',atpos)
>>> print(spos)
31
>>> host = data[atpos+1:spos]
>>> print(host)
uct.ac.za
```

```
>>>
```

Використано версію методу `find`, яка дозволяє вказати позицію в рядку, з якої потрібно почати пошук. Під час зрізу вилучаються символи, починаючи з «символів після @ і закінчуючи пробілом, *але не враховуючи його*».

Документація для методу `find` доступна за посиланням:

<https://docs.python.org/library/stdtypes.html#string-methods>.

6.11 Форматування рядків

Форматований рядок (його часто позначають просто як *f-рядок*) дозволяє використовувати вирази Python у рядкових літералах. Для цього до рядкового літерала додається символ `f`, а вираз береться у фігурні дужки `{}`.

Наприклад, якщо взяти ім'я змінної у фігурні дужки всередині *f-рядка*, воно буде замінено на її значення:

```
>>> camels = 42
>>> f'{camels}'
'42'
```

Результатом буде рядок `'42'`, його не слід плутати з цілим числом 42.

Вираз може з'являтися в будь-якому місці рядка, тому можна вставити значення у речення:

```
>>> camels = 42
>>> f'Я помітив {camels} верблюди.'
'Я помітив 42 верблюди.'
```

Кілька виразів можуть бути включені в один рядковий літерал для створення складніших рядків.

```
>>> years = 3
>>> count = .1
```

РОЗДІЛ 6. РЯДКИ

```
>>> species = 'верблюди'
>>> f'За {years} роки я побачив {count} {species}.'
'За 3 роки я побачив 0.1 верблюди.'
```

Форматовані рядки мають безліч можливостей і можуть робити навіть більше, ніж описано у цій статті. Детальніше за посиланням:

<https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals>.

6.12 Налагодження програми

Навичка, яку варто розвинути під час програмування – завжди запитувати себе: «Що може піти не так?» або, навпаки, «Яку дурницю може втнути користувач, що зламає мою (здавалося б) ідеальну програму?».

Наприклад, розглянемо програму, яку було використано для демонстрації циклу `while` у розділі «Ітерація»:

```
while True:
    line = input('> ')
    if line[0] == '#':
        continue
    if line == 'завершено':
        break
    print(line)
print('Завершено')
```

Код: <https://www.py4e.com/code3/copytildone2.py>

Ось, що відбувається, коли користувач вводить порожній рядок вводу:

```
> привіт
привіт
> # не виводь це на
екран
> виводь ось це!
```

Виводь ось це!

>

Traceback (most recent call last):

File "copytildone.py", line 3, in <module>

if line[0] == '#':

IndexError: string index out of range

Код працює відмінно доти, доки не з'являється порожній рядок. У такому разі нульовий символ відсутній, і з'являється помилка. Існує два способи зробити третій рядок «безпечним», навіть якщо він порожній.

if line.startswith('#):

Інший спосіб – просто записати інструкцію **if** за допомогою запобіжного шаблону і переконатися, що другий логічний вираз обчислюється лише там, де в рядку є хоча б один символ:

if len(line) > 0 **and** line[0] == '#':

6.13 Словник

counter (лічильник) Змінна, призначена для підрахунку чого-небудь, зазвичай ініціалізується нулем, а потім збільшується.

empty string (порожній рядок) Рядок без символів довжиною 0, поданий двома лапками.

flag (прапорець) Булева (логічна) змінна, яка використовується, щоб вказати, чи умова **true** (істинна) чи **false** (хибна).

invocation (виклик) Інструкція, яка викликає метод.

immutable (незмінний) Властивість послідовності, елементи якої не можуть бути присвоєні.

index (індекс) Ціле значення, яке використовується для вибору елемента в послідовності, наприклад, символу в рядку.

item (елемент) Одне зі значень у послідовності.

method (метод) Функція, яка асоціюється з об'єктом і викликається за допомогою запису з крапкою.

object (об'єкт) Те, на що може посилатися змінна. Наразі, поняття «об'єкт» і «значення» ви можете використовувати як взаємозамінні.

РОЗДІЛ 6. РЯДКИ

search (пошук) Шаблон перебору, який зупиняється, коли знаходить потрібне.

sequence (послідовність) Впорядкований набір, тобто сукупність значень, де кожне з них має цілий індекс.

slice (зріз) Частина рядка, задана діапазоном індексів.

traverse (перебирати) Ітерувати елементи у послідовності, виконуючи аналогічну операцію над кожним з них.

6.14 Вправи

Вправа 5: Зріз рядків

Візьміть наведений нижче код Python, який містить рядок:

```
str = 'X-DSPAM-Confidence: 0.8475'
```

Використайте `find` та зріз рядка, щоб виокремити частину рядка після символу двокрапки, а потім скористайтеся функцією `float`, щоб перетворити виокремлений рядок на число з рухомою крапкою.

6.14. ВПРАВИ

Вправа 6: Методи рядків

Прочитайте документацію до методів рядків за посиланням

<https://docs.python.org/library/stdtypes.html#string-methods>.

Можете поекспериментувати з деякими з них, щоб переконатися, що розумієте, як вони працюють. Особливо ефективними є `strip` та `replace`.

У документації використовується синтаксис, який може заплутати вас. Наприклад, у `find(sub[, start[, end]])` дужки вказують на необов'язкові аргументи. Тобто `sub` є обов'язковим, `start` – ні, а якщо вказати `start`, то `end` буде необов'язковим.